Walmart is the biggest retail store in the United States. Just like others, they have been expanding their e-commerce part of the business. By the end of 2022, e-commerce represented a roaring $80 billion in sales, which is 13% of total sales of Walmart. One of the main factors that affects their sales is public holidays, like the Super Bowl, Labour Day, Thanksgiving, and Christmas.

In this project, you have been tasked with creating a data pipeline for the analysis of supply and demand around the holidays, along with conducting a preliminary analysis of the data. You will be working with two data sources: grocery sales and complementary data. You have been provided with the `grocery_sales` table in `PostgreSQL` database with the following features:

`grocery_sales`

- `"index"` - unique ID of the row
- `"Store_ID"` - the store number
- `"Date"` - the week of sales
- `"Weekly_Sales"` - sales for the given store

Also, you have the `extra_data.parquet` file that contains complementary data:

`extra_data.parquet`

- `"IsHoliday"` - Whether the week contains a public holiday - 1 if yes, 0 if no.
- `"Temperature"` - Temperature on the day of sale
- `"Fuel_Price"` - Cost of fuel in the region
- `"CPI"` – Prevailing consumer price index
- `"Unemployment"` - The prevailing unemployment rate
- `"MarkDown1"`, `"MarkDown2"`, `"MarkDown3"`, `"MarkDown4"` - number of promotional markdowns

- `"Dept"` - Department Number in each store
- `"Size"` - size of the store
- `"Type"` - type of the store (depends on `Size` column)

You will need to merge those files and perform some data manipulations. The transformed DataFrame can then be stored as the `clean_data` variable containing the following columns:

- `"Store_ID"`
- `"Month"`
- `"Dept"`
- `"IsHoliday"`
- `"Weekly_Sales"`
- `"CPI"`
- "`"Unemployment"`"

After merging and cleaning the data, you will have to analyze monthly sales of Walmart and store the results of your analysis as the `agg_data` variable that should look like:

Month	Weekly_Sales
1.0	33174.178494
2.0	34333.326579
...	...

Finally, you should save the `clean_data` and `agg_data` as the csv files.

It is recommended to use `pandas` for this project.

Projects Data DataFrame as `groce`

```
-- Write your SQL query here
SELECT * FROM grocery_sales
```

Hidden output

```
import pandas as pd
import os

# Extract function is already implemented for you
def extract(store_data, extra_data):
    extra_df = pd.read_parquet(extra_data)
    merged_df = store_data.merge(extra_df, on = "index")
    return merged_df

# Call the extract() function and store it as the "merged_df" variable
merged_df = extract(grocery_sales, "extra_data.parquet")
```

```
# Create the transform() function with one parameter: "raw_data"
def transform(raw_data):
  # Fill NaNs using mean since we are dealing with numeric columns
  # Set inplace = True to do the replacing on the current DataFrame
    raw_data.fillna(
      {
          'CPI': raw_data['CPI'].mean(),
          'Weekly_Sales': raw_data['Weekly_Sales'].mean(),
          'Unemployment': raw_data['Unemployment'].mean(),
      }, inplace = True
    )

    # Define the type of the "Date" column and its format
    raw_data["Date"] = pd.to_datetime(raw_data["Date"], format = "%Y-%m-%d")
    # Extract the month value from the "Date" column to calculate monthly sales
later on
    raw_data["Month"] = raw_data["Date"].dt.month

    # Filter the entire DataFrame using the "Weekly_Sales" column. Use .loc to
access a group of rows
    raw_data = raw_data.loc[raw_data["Weekly_Sales"] > 10000, :]

    # Drop unnecessary columns. Set axis = 1 to specify that the columns should be
removed
    raw_data = raw_data.drop(["index", "Temperature", "Fuel_Price", "MarkDown1",
"MarkDown2", "MarkDown3", "MarkDown4", "MarkDown5", "Type", "Size", "Date"], axis =
1)
    return raw_data
```

```
# Call the transform() function and pass the merged DataFrame
clean_data = transform(merged_df)
```

```
# Create the avg_weekly_sales_per_month function that takes in the cleaned data from
the last step
def avg_weekly_sales_per_month(clean_data):
    # Select the "Month" and "Weekly_Sales" columns as they are the only ones needed
for this analysis
    holidays_sales = clean_data[["Month", "Weekly_Sales"]]
    # Create a chain operation with groupby(), agg(), reset_index(), and round()
functions
    # Group by the "Month" column and calculate the average monthly sales
    # Call reset_index() to start a new index order
    # Round the results to two decimal places

    holidays_sales = (holidays_sales.groupby("Month")
    .agg(Avg_Sales = ("Weekly_Sales", "mean"))
    .reset_index().round(2))
    return holidays_sales
```

```
# Call the avg_weekly_sales_per_month() function and pass the cleaned DataFrame
agg_data = avg_weekly_sales_per_month(clean_data)
```

```
# Create the load() function that takes in the cleaned DataFrame and the aggregated
one with the paths where they are going to be stored
def load(full_data, full_data_file_path, agg_data, agg_data_file_path):
    # Write your code here
# Save both DataFrames as csv files. Set index = False to drop the index columns
    full_data.to_csv(full_data_file_path, index = False)
    agg_data.to_csv(agg_data_file_path, index = False)
```

```
# Call the load() function and pass the cleaned and aggregated DataFrames with their
paths
load(clean_data, "clean_data.csv", agg_data, "agg_data.csv")
```

```
# Create the validation() function with one parameter: file_path - to check whether
the previous function was correctly executed
def validation(file_path):
    # Write your code here
    # Use the "os" package to check whether a path exists
    file_exists = os.path.exists(file_path)
    # Raise an exception if the path doesn't exist, hence, if there is no file found
on a given path
    if not file_exists:
        raise Exception(f"There is no file at the path {file_path}")
```

```
# Call the validation() function and pass first, the cleaned DataFrame path, and
then the aggregated DataFrame path
validation("clean_data.csv")
validation("agg_data.csv")
```